

Efficiency (and how to obtain it)

Applied programming (LIS050)

Robert Östling `robert@ling.su.se`

Department of Linguistics
Stockholm University

2026-02-03



Stockholm
University

Why efficiency?

- Digital devices use about 5% of the global electricity production, and cause about 2–3% of greenhouse gas emissions
- Several ways your program can consume resources:
 - Direct energy usage (relatively easy to compute:
<https://calculator.green-algorithms.org/>)
 - Production of computing hardware (negligible in data centers, but applies to consumer devices)
 - Worst case scenario: your program needlessly forces users to buy new hardware



A fundamental conflict of interest

Making your program more efficient...

- **costs** time and effort for **you**
- **saves** energy for **someone else**
- **reduces** environmental damage for **someone else** (mostly)



Additional problems

- If a user can accept a delay of 1 second, does it matter whether your program runs for 0.5 seconds or 0.005 seconds?
 - Not immediately, but if another just-below-the-limit program is installed the system could start feeling “slow”
 - How much computation will your program perform **in total** over its lifetime for all users?



Additional problems

- If a user can accept a delay of 1 second, does it matter whether your program runs for 0.5 seconds or 0.005 seconds?
 - Not immediately, but if another just-below-the-limit program is installed the system could start feeling “slow”
 - How much computation will your program perform **in total** over its lifetime for all users?
- Monetary cost may not match the environmental cost, your (expensive) time will be weighed against:
 - Energy costs. Worst case: subsidized fossil fuels
 - Hardware costs. Worst case: materials extracted by low-wage miners in mines somewhere with lax environmental regulations



How to make programs more efficient?

Usually you try the following, in order:

- 1 Solve a different problem.
- 2 Use more efficient algorithms.
- 3 Optimize your implementation of the algorithms.



Solve a different problem

Here I mean *technically* different problems, even if the end result to the human is roughly the same.

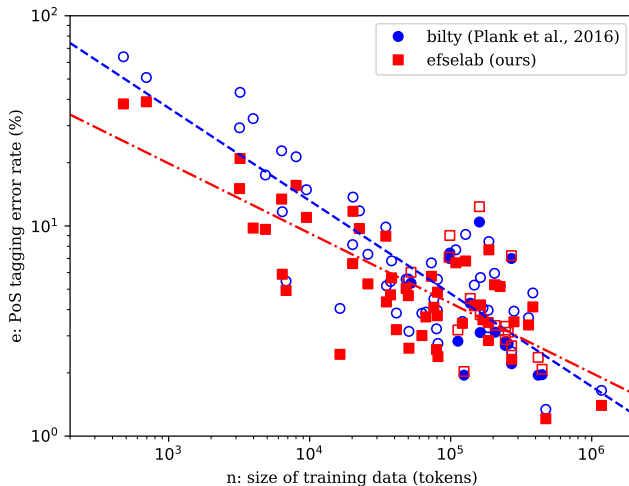
The most recent models are not always the most suitable. Let us look at the problem of part of speech tagging:

- Many methods can be used: rule-based, feature-based ML, traditional neural networks, LLMs, etc.
- Plank et al. (2016) showed that a (then) advanced neural network model had state-of-the-art performance across 22 languages
- Unfortunately, quite resource-intensive (by the standards of that time)
- I compared to an old-fashioned feature-based model on 48 languages



Part of speech tagging

efselab is ≈ 750 times faster than bilty, but accuracy is similar



Adjust your expectations

- What level of accuracy can you accept?
- How much extra computation is a given amount of improvement worth?



Use a more efficient algorithm

Classic example, sorting items (say: a pile of numbered pages, out of order)

- Simple method: flip through the pile and if you see two adjacent pages in the wrong order, reverse their order. Repeat until everything is in the right order.



Use a more efficient algorithm

Classic example, sorting items (say: a pile of numbered pages, out of order)

- Simple method: flip through the pile and if you see two adjacent pages in the wrong order, reverse their order. Repeat until everything is in the right order.
- What better ways are there to solve **exactly the same problem**?



Use a more efficient algorithm

Classic example, sorting items (say: a pile of numbered pages, out of order)

- Simple method: flip through the pile and if you see two adjacent pages in the wrong order, reverse their order. Repeat until everything is in the right order.
- What better ways are there to solve **exactly the same problem**?
- Bonus question: what *worse* methods are there?



Analysis of algorithms

- This is a whole research field, we will only discuss the basics
- Computer scientists ask “*how do the computational resources consumed grow as the size of the input goes towards infinity?*”
- What you typically **want** to know is “*how long will it take to run for the kinds of data that users of my program will present to it?*”
- If you make your code 1000 times faster for any given input, this is of great practical importance, but a computer scientist will say the computational complexity is the same.
- Still, usually a good idea to choose algorithms with lower complexity



Big O notation

Describes the **order of growth**. Some examples, where n is always the size of the input:

Time	Order
1	$O(1)$
1 000 000	$O(1)$
n	$O(n)$
$4n + 7$	$O(n)$
$4n + \sqrt{n} + 7$	$O(n)$
$4n \log n + \sqrt{n} + 7$	$O(n \log n)$
$n^2 + 4n + \sqrt{n} + 7$	$O(n^2)$
2^n	$O(2^n)$
$100 \times 2^n + n^2 + 4n + \sqrt{n} + 7$	$O(2^n)$



Complexity of built-in Python operations

set/dict membership	$O(1)$
list membership	$O(n)$
insert in set/dict	$O(1)$
insert in list	$O(n)$
append to list	$O(1)$
insert in heapq	$O(\log n)$
min() of list	$O(n)$
min() of set	$O(n)$
min() of heapq	$O(1)$
sorted()	$O(n \log n)$



An NLP example

Dependency grammar parsing algorithms (n = length of sentence)

- Collins (1996): $O(n^5)$ — “over 200 sentences per minute”
- Eisner (1996): $O(n^3)$
- McDonald et al. (2005): $O(n^2)$
- Nivre (2003): $O(n)$



An NLP example

Dependency grammar parsing algorithms (n = length of sentence)

- Collins (1996): $O(n^5)$ — “over 200 sentences per minute”
- Eisner (1996): $O(n^3)$
- McDonald et al. (2005): $O(n^2)$
- Nivre (2003): $O(n)$

There are some differences in what exact problem these algorithms solve, but think for a moment about the implications if you are interested in parsing a text corpus:

- What is the distribution of sentence lengths?
- What might you encounter in the future?
- What are the constant factors?



- How to express your algorithm in a given programming language?
- (Antique) example: $x + x$ might be faster to compute than $2x$
- Python generally takes care of this kind of detail for you
- There are still ways to speed up code:
 - Lower-level code (Numba, Cython, C, etc.)
 - Vectorization (with NumPy, pyTorch, etc.)
- Note that vectorization often results in speedup by parallelization, which may reduce the amount of **time** spent waiting for an answer, but not the total amount of **computation** (and energy)



Efficient algorithms

- Another huge subfield of computer science
- We make a tiny selection with a focus on NLP applications
- Some tasks (e.g. sorting) is almost always done by Python or libraries
- Other tasks are too specialized to discuss here



The Levenshtein algorithm

Our task: find the smallest number of edit operations (insert, delete, replace) required to transform string x to string y .

		s	a	y
	0	1	2	3
p	1	1	2	3
l	2	2	2	3
a	3	3	2	3
y	4	4	3	2



Dynamic programming

- The Levenshtein algorithm is an instance of a more general family of algorithms: Dynamic programming
- The key is to save computation by finding a way to reuse previous computations of sub-problems
- $L(x_{1\dots n}, y_{1\dots m})$ can easily be computed from $L(x_{1\dots n-1}, y_{1\dots m})$, $L(x_{1\dots n}, y_{1\dots m-1})$ and $L(x_{1\dots n-1}, y_{1\dots m-1})$
- Let us look at a simple implementation...



Minimal edit distance, the stupid way

```
def L(x, y):  
    if len(x) == 0:  
        return len(y)  
    elif len(y) == 0:  
        return len(x)  
    else:  
        replace_cost = 0 if x[-1] == y[-1] else 1  
        return min(1 + L(x[:-1], y),  
                    1 + L(x, y[:-1]),  
                    replace_cost + L(x[:-1], y[:-1]))
```



Applications of dynamic programming

Other NLP problems that are often solved using dynamic programming algorithms:

- Part of speech tagging, Named Entity Recognition, etc. (the Viterbi algorithm)
- Syntactic parsing (e.g. the CKY algorithm)
- Word alignment in parallel texts (Vogel 1996)
- Word segmentation



What is Numba?

- Tool for just-in-time (JIT) compilation of Python code
- Normal Python: interpret each function
- With `@njit` from Numba:
 - First call only: Numba **compiles** your function to fast machine code
 - Every call: The Numba-generated machine code is used, instead of the Python interpreter
- Why not use `@njit` everywhere?
 - 1 Numba itself requires time and memory, this investment does not always pay off.
 - 2 Numba can only handle certain Python constructs.



When does Numba help?

These are some typical examples of structures that can be helped by Numba:

- Loops with integer indices (`for i in range(n): ...`)
- Basic arithmetic operations and indexing (`x[i] = y[i] * z[i]`)
- Many iterations/recursions of the same code
- Homogeneous data structures (lists of ints or floats, numpy arrays)
- Vector operations with NumPy



When does Numba not help?

There are some common cases where Numba only makes code slower, or will not work at all:

- Calls to external libraries (except NumPy)
- Heterogeneous data structures (list, dicts, etc. with elements of different types)
- File/network operations

Be prepared to tweak your code to Numba's liking!



(Naive) edit distance, first attempt

L("Levenshtein", "Neuschwanstein"): 177 $\rightarrow$ 95 seconds

```
@njit
def L(x, y):
    if len(x) == 0:
        return len(y)
    elif len(y) == 0:
        return len(x)
    else:
        replace_cost = 0 if x[-1] == y[-1] else 1
        return min(1 + L(x[:-1], y),
                    1 + L(x, y[:-1]),
                    replace_cost + L(x[:-1], y[:-1]))
```



Problems with the first attempt

- Repeated string operations (slicing, indexing, comparisons)
- The only thing we really need the strings for is to check for equality between letters
- We first compute a distance matrix:
distance[i][j] is 0 if $x[i] == y[j]$, otherwise 1
- We then write a function using only indexes and the distance matrix



Getting rid of the strings

`L("Levenshtein", "Neuschwanstein")`: 98 $\rightarrow$ 38 seconds

Note: this helps even without `@njit`, and the Numba speedup factor increases as well

```
@njit
def Lrec(distance, i, j):
    if i == 0:
        return j
    if j == 0:
        return i
    return min(1 + Lrec(distance, i-1, j),
               1 + Lrec(distance, i, j-1),
               distance[i-1][j-1]
               + Lrec(distance, i-1, j-1))
```



Getting rid of the strings

Minor complication: Numba can not handle nested lists because it can not verify that they are homogeneous, but if we use its own List class it can figure this out.

```
from numba.typed import List
```

```
# Regular lists when Lrec does not have @njit
```

```
def L(x, y):  
    distance = [[int(xc != yc) for yc in y] for xc in x]  
    return Lrec(distance, len(x), len(y))
```

```
# Must use this when Lrec has @njit
```

```
def L(x, y):  
    distance = List([List([int(xc != yc) for yc in y])  
                    for xc in x])  
    return Lrec(distance, len(x), len(y))
```



Introducing NumPy

`L("Levenshtein", "Neuschwanstein")`: 164 $\rightarrow$ 6 seconds

Note: non-vectorized NumPy code is slow without Numba, but becomes 27 times faster with `@njit`

```
@njit
def Lrec_np(distance, i, j):
    if i == 0:
        return j
    if j == 0:
        return i
    return min(1 + Lrec_np(distance, i-1, j),
               1 + Lrec_np(distance, i, j-1),
               distance[i-1, j-1]
               + Lrec_np(distance, i-1, j-1))
```



Best of all: using a good algorithm

L("Levenshtein", "Neuschwanstein"): 0.01 seconds

```
def L(x, y):
    m = [[None]*(len(y)+1) for _ in range(len(x)+1)]
    for i in range(len(x)+1):
        m[i][0] = i
    for j in range(len(y)+1):
        m[0][j] = j
    for i in range(1, len(x)+1):
        for j in range(1, len(y)+1):
            replace_cost = int(x[i-1] != y[j-1])
            m[i][j] = min(1 + m[i-1][j],
                          1 + m[i][j-1],
                          replace_cost + m[i-1][j-1])
    return m[-1][-1]
```



Numba-ize the Levenshtein algorithm and see how fast you can get it.
Some hints:

- Check the Numba 5-minute introduction, section “How to measure the performance of Numba?” for hints on how to time code accurately.
- Time your implementation with inputs of different length. Does it scale as the product of the input sizes as expected, i.e. $O(n \times m)$?
- Is there any size of input where the naive algorithm from previous slides is better?

